

Resource Management & the Rule of Three

CSCI 235 — Class #5

Tong Yi

Hunter College, CUNY

Tuesday, September 15, 2026

From Your Lecture Slips

Answers to the questions you turned in on Thursday (9/10),
grouped by topic.

What Does “Amortized” Mean, in Detail?

Your questions

“Clarification on amortization” / “More examples on what amortized means?” / “I still don’t know what Amortized is in detail.”

Amortized cost: spread the expensive operations over *all* the operations, and look at the **average**.

An everyday analogy

A \$1,200 textbook costs \$1,200 the month you buy it — but *amortized* over a 12-month course, it’s \$100/month. Same idea: most `push_back` calls cost nothing extra; a rare few cost a full copy — averaged over all the pushes, the cost per push is still bounded.

Concretely

Pushing 9 elements: pushes 3, 5, 9 copy 2, 4, 8 elements (every other push copies 0). Total = 14 copies. Amortized = $14/9 \approx 1.6$ per push — bounded, even though push 9 alone cost 8.

Nested Loops: Yes, That's $O(n) \times O(n) = O(n^2)$

Your question

“If you have a nested loop, would that be $O(n) \times O(n)$? Then that's $O(n^2)$?”

Exactly right. Each pass of the outer loop triggers a *full* pass of the inner loop — for example, printing the $n \times n$ multiplication table:

```
for (int i = 1; i <= n; ++i) {           // n times
    for (int j = 1; j <= n; ++j) {       // n times, EACH outer pass
        std::cout << std::setw(4) << i * j; // O(1) work: one product
    }
    std::cout << '\n';
}
```

The rule

Every one of the n rows prints n products, so the total work is $n \times n = n^2$ print statements — the loop runs in $O(n^2)$. In general: nesting a loop that runs $O(f(n))$ times inside one that runs $O(g(n))$

Visualizing $O(n^2)$: One Print per Cell

For $n = 4$, the nested loop visits every pair (i, j) with $i, j \in [1, 4]$ exactly once — one cout per cell of a 4×4 grid:

		j (inner loop) $\rightarrow$			
		1	2	3	4
i (outer loop) $\rightarrow$	1	1	2	3	4
	2	2	4	6	8
	3	3	6	9	12
	4	4	8	12	16

The count

4 rows $\times$ 4 columns = 16 cells — in general, n rows $\times$ n columns = n^2 cells, one unit of work (one cout) per cell. **That's exactly what $O(n^2)$ means:** not a formula to memorize, but “the work grows like the area of a grid that scales with n in both directions.”

$O(\log n)$, in More Depth

Your question

“Can you go over $O(\log n)$ more in-depth?”

$O(\log n)$ shows up whenever each step **cuts the remaining work in half** (binary search; the guessing game from last class). If you can only halve k times before reaching 1 element, then $2^k = n$, so $k = \log_2 n$.

n	$\lceil \log_2 n \rceil$ (max steps)
100	7
1,000	10
1,000,000	20

Why this matters

Even a *million* elements take only about 20 halvings — doubling n costs just **one more** step, not double the work. That's what makes $O(\log n)$ grow so much more slowly than $O(n)$.

Big-O Is a Notation, Not a Mystery Symbol

Your questions

“I know Big-O is for growth rates, but does the O itself mean anything specific, or is it just the notation? Is there a direct way to confirm something's Big-O growth rate if you're unsure?”

O stands for “**order of**” — notation for “bounded by a constant multiple of, for large n .” $O(n)$ isn't a special value; it names a *family* of functions that all grow at most linearly.

Finding it analytically

Count how the **dominant operation** scales with n : one loop $\rightarrow O(n)$; a nested loop $\rightarrow O(n^2)$; halving each step $\rightarrow O(\log n)$; constant work $\rightarrow O(1)$.

Checking it empirically

Time the code at n , then $2n$. Roughly the same time suggests $O(1)$; roughly doubled suggests $O(n)$; roughly quadrupled suggests $O(n^2)$; barely longer suggests $O(\log n)$.

`new int[n]{}:` Why the Curly Braces?

Your question

“When initializing `data_`, why are there curly brackets after the ‘`new int[...]`’?”

```
int* a = new int[5]; // UNINITIALIZED -- garbage values
int* b = new int[5]{}; // VALUE-INITIALIZED -- all zeros
```

Why it matters for MiniVector

Without `{}`, the new slots hold whatever bytes were already sitting in memory — reading them before writing is undefined behavior. The empty `{}` guarantees every element starts at a known value (0 for `int`).

After `delete[] data_;`, What Happens to `data_` Itself?

Your question

“When using `delete`, does it get rid of what was being pointed to forever, while the pointer itself still exists, without pointing to anything anymore?”

Exactly right, in two separate parts:

- `delete[] data_;` frees the **memory** the array occupied — that memory is released.
- The **pointer variable** `data_` itself is untouched by that statement — it still holds the *old address*, now pointing at freed memory. That’s called a **dangling pointer**.

This is exactly why

`reserve()` immediately reassigns `data_` to the new array right after freeing the old one — so it’s never left dangling (recall “Destructors: Why No `data_ = nullptr;`?” from last class).

Reallocation vs. Copying: Two Different Steps

Your questions

“What’s the difference between reallocation and existing elements copied?” / “When do we copy elements? Is it when we increase the capacity?”

During `push_back()`, copying existing elements only happens *as part* of a reallocation, but reallocation and copying are two distinct actions:

- 1 **Reallocation:** `new int[newCapacity]` — obtaining a *new*, bigger block of memory.
- 2 **Copying:** moving each existing element from the old array into the new one, one at a time.

When does this happen?

Only when `push_back` finds `size_ == capacity_` — the array is full. A normal push into a non-full array does **neither**: no reallocation, no copying, just one assignment.

The Capacity Math, One Step at a Time

Your questions

“I don’t understand the math behind the expansion of the capacity...” / “I get how it grows but not the computation — where does the m come from?”

Let m just **count reallocations** — the 1st, 2nd, 3rd time the array has doubled. Starting from `capacity_ = 2`:

m	happens at push	copies this time	capacity becomes
1	3	2	4
2	5	4	8
3	9	8	16

The pattern

Reallocation m happens at push $2^m + 1$ and copies 2^m elements — m is simply *which doubling we’re on*, nothing more exotic. That’s the m in $2 + 4 + \dots + 2^m = 2^{m+1} - 2$ (full derivation: Day 4 backup slides).

Where We Are

- MiniVector **owns** a heap array: `data_` points to memory it allocated with `new[]`, and its destructor frees that memory with `delete[]`.
- We've built the parts that make it **grow** (`reserve()`, `push_back()`) and we can measure how *fast* it grows (Big-O, amortized $O(1)$).
- One piece is still open — **Task D**: what happens when you *copy* a MiniVector?

Today

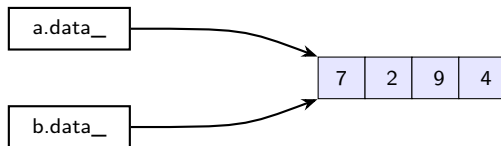
MiniVector `m2 = m1`; and `m2 = m1`; both compile right now — but they're quietly **broken**. Today we find out why, and fix them for good.

Recap: The Shallow-Copy Danger

Suppose we never write a copy constructor. The compiler generates one for free — it copies **every member, one field at a time**:

```
MiniVector b = a; // compiler-generated: member-wise copy
```

For `size_` and `capacity_` (plain numbers) that's exactly right. For `data_` (a **pointer**) it copies the *address* — not the array it points to:



Two objects, one array

a and *b* now both believe they own this array. Neither one does, safely.

Recap: Why That's a Problem

Once a and b share one array:

- Modifying b's elements silently changes a's too — they're the *same* memory.
- When a and b both go out of scope, *both* destructors run `delete[] data_;` on the **same address** — a **double free**, undefined behavior.
- If a is destroyed first, `b.data_` is left **dangling** — pointing at freed memory.

The real issue

A member-wise copy is exactly right for members that *are* their own value (like `ints`). It's exactly **wrong** for a member that's a *pointer to a resource you own*.

What Is a “Resource”?

A **resource** is anything you must explicitly *acquire* and, sooner or later, explicitly *release*:

- Heap memory — acquired with `new[]`, released with `delete[]`.
- A file — opened, then must be closed.
- A network connection, a database lock, ... acquire, then release — every time.

MiniVector's resource is its heap array. **Owning** a resource means being responsible for releasing it — exactly once, no more, no less.

An Analogy: The Library Checkout Slip

Think of `data_` as a **checkout slip**, not the book itself.

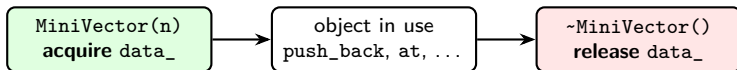
- Checking out a book (`new[]`) gives you a slip recording *which* book is yours.
- Returning it (`delete[]`) hands the book back — your slip is now meaningless.
- If your roommate **photocopies your slip**, they hold a slip for the *same* book — not a book of their own. If you return it, their slip now points at a book that's back on the shelf.

The fix

A correct copy doesn't copy the slip — it checks out a **second, identical book**. That's a **deep copy**.

RAII: Resource Acquisition Is Initialization

The idiom C++ uses to manage resources safely: tie the resource's lifetime to an **object's** lifetime, so the *language itself* — not the programmer's memory — decides when the resource is released.



Why it matters

You don't need to remember to release the resource manually when the object's lifetime ends — its destructor runs automatically, no matter how execution gets there. But this guarantee only holds if **copying** the object is also handled correctly — which brings us to ...

RAII in Action: No Manual Cleanup

Compare a `MiniVector` to a resource that needs *manual* release, like a raw `FILE*` you must `fclose` yourself:

```
void useVector() {  
    MiniVector v(5); // constructor: ACQUIRE data_  
    v.push_back(10);  
    v.push_back(20);  
    // ... use v ...  
} // scope ends --> destructor: RELEASE data_, automatically
```

The key difference

You never write a matching “cleanup” call yourself. Whether `useVector` returns normally, returns *early*, or an exception is thrown partway through, `v` still goes out of scope — so `~MiniVector()` *still* runs, and `data_` is still freed.

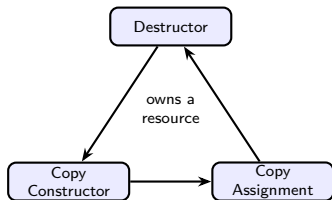
Without RAII

A resource managed by hand (e.g. a bare `malloc/free` pair with no destructor) needs a matching release call on **every** exit path — miss one and it leaks.

The Rule of Three

If you need to write one, you need all three

If a class needs a hand-written **destructor**, it almost certainly also needs a hand-written **copy constructor** and **copy assignment operator**.



Member	Written?
Destructor	✓ (Day 4)
Copy ctor	today
operator=	today

Why they travel together: a custom destructor signals the class owns a resource the compiler can't copy for you — so its default copying is wrong for the same reason its default destructor would be: it would not delete[] the heap array owned through data_.

The Copy Constructor: When Is It Called?

Recall from Day 3: it runs whenever a **new** object is being *created from* an existing one.

```
MiniVector a(5);  
MiniVector b = a;    // b is being CREATED -> copy constructor  
MiniVector c(a);     // same thing, direct syntax  
void f(MiniVector v); // pass-by-value -> copy constructor
```

Contrast with operator=

`a = b`; is different: `a` already **exists**, so this calls the copy **assignment** operator instead (coming up next).

Writing the Copy Constructor

Takes a `const` reference to the object being copied, and builds a **deep** copy — fill in the blanks together in class:

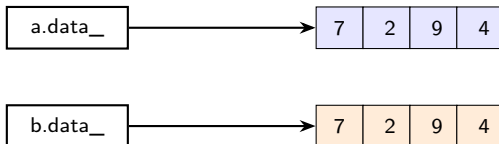
```
MiniVector::MiniVector(const MiniVector& other)
: data_(/* 1. allocate ??? ints */),
  size_(/* 2. copy other's size_ */),
  capacity_(/* 2. copy other's capacity_ */) {
    for (std::size_t i = 0; i < /* loop bound */; ++i) {
        /* 3. copy ONE element -- not the pointer */
    }
}
```

Key steps

1. Allocate its own array, sized to other's capacity.
2. Copy `size_` and `capacity_`.
3. Copy each *element*, one at a time.

Two Separate Books, Same Contents

Now `a` and `b` each own their **own** array:



- Modifying `b` never touches `a`; each destructor frees its *own* array — no double free.

Why `other.capacity_`, not `other.size_`?

We allocate room for `other`'s full `capacity_` so the copy's growth behavior matches the original — though allocating just `size_` would also be a valid, simpler design choice.

operator=: When Is It Called?

It runs when an **already-existing** object is assigned a new value:

```
MiniVector a(5), b(3);  
a = b;    // a already EXISTS -> copy assignment operator
```

Its signature returns a **reference** to itself, so assignments can chain (a = b = c;):

```
MiniVector& operator=(const MiniVector& other);
```

The Self-Assignment Trap

A tempting first attempt:

```
MiniVector& MiniVector::operator=(const MiniVector& other) {  
    delete[] data_;           // free our old array  
    data_ = new int[other.capacity_]; // allocate a new one  
    for (std::size_t i = 0; i < other.size_; ++i)  
        data_[i] = other.data_[i];    // copy from other  
    size_ = other.size_;  
    capacity_ = other.capacity_;  
    return *this;  
}
```

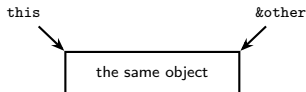
What if `a = a;`?

Then `other` refers to the *same object* as `*this`. Line 1 destroys the original array. After line 2 assigns `data_` a new array, `other.data_` refers to that *same new* array too — so the loop reads elements from the new array *before* they've been initialized: undefined behavior. The original values are already lost.

The Guard: Check for Self-Assignment

The fix is one line at the top, comparing **addresses**:

```
MiniVector& MiniVector::operator=(const MiniVector& other) {  
    if (this == &other) {  
        return *this; // assigning to myself: nothing to do  
    }  
    // ... proceed with the real assignment below  
}
```



this and &other are both MiniVector*. They're equal exactly when a = a; — the same object on both sides.

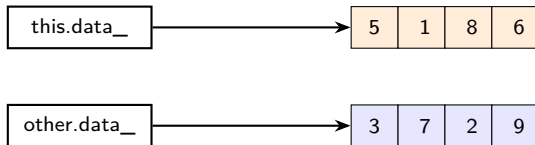
Writing operator=: What It Must Do

No fill-in-the-blank this time — write the whole function yourselves. Here is exactly what each of the five steps must accomplish:

- ❶ **Guard.** If `other` is `*this`, do nothing further — return immediately.
- ❷ **Allocate.** Get a brand-new array sized to `other`'s capacity. Don't touch your own array yet.
- ❸ **Copy.** Copy each of `other`'s `size_` elements into the new array, one at a time.
- ❹ **Free the old.** Only *now* release the array you currently own.
- ❺ **Point and update.** Point `data_` at the new array, and copy `other`'s `size_` and `capacity_` into your own. Return `*this` by reference.

Before operator=: Two Independent Owners

this and other are already separate, valid objects — each owns its own array, with its own values:

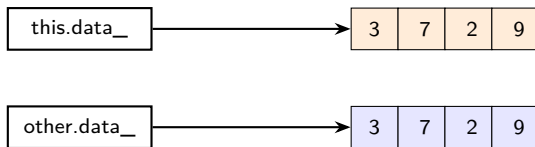


The goal

After `a = b;` finishes, `a` should hold its *own* copy of `b`'s values — without ever being left without a valid array along the way.

After operator=: Same Values, Still Independent

Once the five steps finish, `this` owns a **new** array holding `other`'s values — `other` is completely untouched, and `this`'s *old* array has been freed:

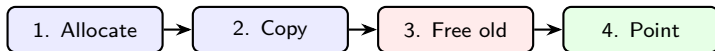


Why the order matters

Steps 2–3 (allocate, copy) happen *before* step 4 (free). That ordering is what makes every step safe to reason about — including a case we'll return to shortly.

Same Pattern as `reserve()`!

The exact four steps `reserve()` uses to grow the array (Day 4)
— **allocate, copy, free, point**:



Why allocate before you free?

If `new int[...]` runs out of memory and throws, we haven't touched `data_` yet — the object we're assigning *into* is left unchanged, rather than half-destroyed. Never give up what you have until the replacement is ready.

So... Is the Guard Actually Necessary?

Trace `a = a;` through this ordering **without** the guard:

- 1 Allocate a *new* array — a different address from `a.data_`.
- 2 Copy `other.data_[i]` into it. *other* is `a`, but `a.data_` hasn't been freed yet, so every value read here is still valid.
- 3 Now free the old array — every value we needed was already copied out in the step above.
- 4 Point `data_` at the new array; update `size_` and `capacity_` (plain numbers, unaffected by freeing memory).

It's correct either way

Because we allocate and copy *before* we free, self-assignment causes no harm — even without the guard.

Then Why Keep the Guard?

Efficiency, not safety

Without the guard, `a = a`; still performs a full allocate + copy + free — an unnecessary $O(n)$ round trip for something that should cost $O(1)$.

The takeaway

For *this* ordering of steps (allocate/copy before free), the self-assignment guard is a **shortcut**, not a safety net — it lets the rare case `a = a`; return in $O(1)$ instead of paying for a full copy it doesn't need.

Contrast this with the *free-then-allocate* version from “The Self-Assignment Trap”: there, the guard **is** required for correctness, since freeing `data_` first destroys the values `other` still needs to read.

The Big Three, Side by Side

Member	Called when	Must do
Destructor	object's lifetime ends	<code>delete[] data_;</code>
Copy constructor	new object created from existing one	allocate, copy elements
Copy assignment	existing object assigned a value	guard, allocate, copy, free old, point

One coherent contract

All three exist to answer the same question consistently: *who owns this heap array, and when is it freed?*

Quick Check

Q1. `MiniVector b = a;` — does this call the copy constructor or operator=?

Quick Check

Q1. `MiniVector b = a;` — does this call the copy constructor or operator=?

Answer

Copy constructor — `b` is being *created*.

Quick Check

Q1. `MiniVector b = a;` — does this call the copy constructor or operator=?

Answer

Copy constructor — `b` is being *created*.

Q2. Why does a double free happen with the compiler's default (member-wise) copy?

Quick Check

Q1. `MiniVector b = a;` — does this call the copy constructor or operator=?

Answer

Copy constructor — `b` is being *created*.

Q2. Why does a double free happen with the compiler's default (member-wise) copy?

Answer

Both copies' `data_` point at the same address; both destructors call `delete[]` on it.

Quick Check

Q1. `MiniVector b = a;` — does this call the copy constructor or operator=?

Answer

Copy constructor — `b` is being *created*.

Q2. Why does a double free happen with the compiler's default (member-wise) copy?

Answer

Both copies' `data_` point at the same address; both destructors call `delete[]` on it.

Q3. With our `allocate/copy/free/point` implementation, why do we still check `this == &other`?

Quick Check

Q1. `MiniVector b = a;` — does this call the copy constructor or operator=?

Answer

Copy constructor — `b` is being *created*.

Q2. Why does a double free happen with the compiler's default (member-wise) copy?

Answer

Both copies' `data_` point at the same address; both destructors call `delete[]` on it.

Q3. With our `allocate/copy/free/point` implementation, why do we still check `this == &other`?

Answer

For efficiency. Without the guard, self-assignment is still correct, but it performs an unnecessary $O(n)$ `allocate/copy/free` operation.

Common Pitfalls

- **Freeing before copying** in `operator=` — makes self-assignment unsafe, because `a = a;` destroys the data before it is copied.
- **Forgetting the self-assignment guard** — not *incorrect* with our allocate-before-free implementation, but wastes $O(n)$ work on `a = a;`.
- **Freeing before allocating** in `operator=` — leaves the object with a dangling `data_` if the new allocation fails.
- **delete instead of delete[]** — mismatching `new[]` with `delete` causes undefined behavior. Always pair `new[]` with `delete[]`.
- **Writing only one or two of the Big Three** — the compiler quietly generates the others, and they'll be wrong in exactly the way this lecture just showed.

What's Due

Item	Posted	Due
Exercise 3 (Big Three)	today, Tu 9/15	Tu 9/22
Project 1A: MiniVector	Tu 9/1	Th 9/17

Reminder

Exercise 2 (Pointer Practice) is due **today**.

Full details and due dates are always on the course schedule page.

Key Takeaways

- A class that **owns a resource** (like heap memory) needs the **Rule of Three**: destructor, copy constructor, copy assignment.
- **RAII** ties a resource's lifetime to an object's lifetime — acquire in the constructor, release in the destructor.
- The compiler's default **member-wise** copy is exactly wrong for a pointer member you own — it copies the address, not the array, causing shared ownership and **double frees**.
- A **deep copy** gives each object its own array with the same values.
- `operator=` should check for self-assignment to avoid unnecessary work, and should **allocate/copy before freeing** the old array — the same allocate/copy/free/point pattern as `reserve()`.

Questions?

See you Thursday, 9/17 — Completing MiniVector: Testing, Debugging & Memory Errors.

Backup Slides

Complete, uncondensed code for reference.

Backup: Copy Constructor, Complete

```
// MiniVector.hpp
class MiniVector {
public:
    MiniVector(const MiniVector& other);
    // ...
private:
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
};

// MiniVector.cpp
MiniVector::MiniVector(const MiniVector& other)
    : data_(new int[other.capacity_]),
      size_(other.size_),
      capacity_(other.capacity_) {
    for (std::size_t i = 0; i < size_; ++i) {
        data_[i] = other.data_[i];
    }
}
```

Backup: operator=, Complete

```
// MiniVector.hpp
class MiniVector {
public:
    MiniVector& operator=(const MiniVector& other);
    // ...
};

// MiniVector.cpp
MiniVector& MiniVector::operator=(const MiniVector& other) {
    if (this == &other) return *this;

    int* newData = new int[other.capacity_];
    for (std::size_t i = 0; i < other.size_; ++i) {
        newData[i] = other.data_[i];
    }

    delete[] data_;
    data_ = newData;
    size_ = other.size_;
    capacity_ = other.capacity_;
    return *this;
}
```

Backup: Tracing `a = a;` With the Bug

Suppose `a.data_` starts at address 500, holding `[7, 2, 9, 4]`, and `operator=` has **no guard**:

Step	What happens
<code>delete[] data_;</code>	The old array at address 500 is destroyed.
<code>data_ = new int[...]</code>	Allocates a fresh address, say 700: <code>a.data_ = 700</code> . Because <code>other</code> is the <i>same object</i> as <code>a</code> , <code>other.data_</code> is 700 too.
<code>data_[i] = other.data_[i]</code>	Reads elements of the <i>new</i> , address-700 array — <i>before</i> they've been initialized: undefined behavior.

The original values `[7, 2, 9, 4]` are already gone by this point — nothing in this trace ever reads them back. The guard `if (this == &other)` `return *this;` sidesteps all of this by doing nothing when it's not needed.